Notty: game engine notes

How the `GameEngine` runs the game, and what I built

COMP4008 group project, University of Nottingham, 2025-26 | Author: Choong Sze Ying | Code: `notty/element_action/game.py`

1. Overview

Notty is a card game for one human and one or two computer players. The player and AI classes define which actions are possible. The `GameEngine` is the controller that runs the whole game around them.

When a game starts, the engine builds and shuffles the deck, creates the human player and the AI opponents, and deals four cards to each. From then on it keeps track of whose turn it is, which actions have been used, checks every move against the rules, and checks for a winner after each one. The engine does not depend on pygame, so it can be tested without a window.

> **My part:** I built the `GameEngine`, and a winner pop-up for the interface. The rest of the interface, the cards and the AI players were written by teammates.

2. A turn at a glance

1. The human player acts: draw, steal, draw one and discard one, or discard a group.
2. `end_player_turn()` checks that any required discard has been done, resets the turn flags and passes the turn on.
3. `run_ai_phase()` lets each AI player take its turn in order. For each one, `ai_take_turn()` asks the AI what to do, carries it out, and checks for a win.
4. Control returns to the human player and the round counter goes up.

3. State the engine keeps

Variable	Purpose
`used_draw`	Number of single draws used this turn (maximum 3).
`used_steal`	Whether the steal action has been used this turn.
`used_draw_discard`	Whether the draw-one-discard-one action has been used this turn.
`pending_discard_required`	Set after the draw step of draw-one-discard-one. The player cannot end the turn until a card is discarded.
`current_index`, `round`	Whose turn it is, and how many rounds have been played.
`deck`, `discard_pile`	The cards still to draw, and the cards that have been discarded.
`winner`	Name of the winning player, or empty while the game is running.

4. Player actions

Action	Methods	Rules the engine enforces
Draw up to three	`player_draw_up_to_three()`	One card per call, at most three per turn. Refused if the hand already has 20 cards or the deck is empty.
Draw one, discard one	`player_draw_one_for_discard()`, `player_discard_one_after_draw()`	Two phases. After the draw, `pending_discard_required` locks the turn until one card is discarded. Once per turn.
Steal	`player_steal()`	Takes a random card from a chosen opponent. Once per turn. Refused if the target has no cards or the player's hand is full.
Discard a group	`player_discard_group()`	Every card must be in the player's hand. The group must be a run of at least 3 cards of one colour with consecutive numbers (`_is_sequence_group`), or at least 4 cards with the same number and all different colours (`_is_same_rank_diff_suit_group`).

Every action returns a result with `ok`, a `reason` and a `message` (plus extra data such as the cards drawn). An illegal move, for example an invalid group or an action already used, therefore comes back as a clear message for the interface instead of an error.

5. Winning

After every move, `_check_winner_after_move()` looks at all players. The first player with an empty hand becomes the `winner`. This applies to the AI as well, so an AI player can win in the middle of its turn.

6. AI players

The engine does not decide the AI's moves itself. `think_action()` on the AI player chooses an action, and `ai_take_turn()` carries it out and updates the hands and piles. `run_ai_phase()` repeats this for each AI player in order. The interface can also run one AI turn at a time with `run_single_ai_turn()`, so each move can be animated.

`set_difficulty()` replaces the AI players with easy or hard ones and restarts the game. `player_auto_play()` asks an AI to choose a move for the human player, which is used as a helper.

7. Deck and discard pile

Discarded cards go to `discard_pile`. `recycle_discard_pile()` moves them back into the deck and shuffles. The engine provides this method, and the interface calls it when the deck runs out, so the card movement can be animated first. `reset_game()` starts a new game: it rebuilds the deck, empties the piles and hands, clears the winner and deals again.

8. Connection to the interface

The interface sends each button click to an engine method, then reads the new state and redraws the screen. `ui_snapshot()` returns the player's hand, the AI hands, the deck, the discard pile, the number of AI players, the current player and the winner. `get_snapshot()` returns the round, the current player, the deck count and the winner for the status bar.

Winner pop-up (my interface part). When the game ends, the screen is darkened and blurred, an animated victory message is shown with a sound, and a click anywhere starts a new game.

9. How I built and tested it

1. I planned the game loop first: set-up, player turns, AI turns, win detection and move checking.
2. I built deck handling, dealing and turn switching, then added the turn flags to stop illegal moves.
3. The hardest part was draw-one-discard-one without breaking the turn order. I solved it with the two-phase flow and `pending_discard_required`.
4. I added group checking for runs and same-number sets, then connected the AI so its turns run automatically.
5. I tested by playing simulated games and fixed edge cases such as the deck running out. The repository includes `simulate_games.py`, which plays full games without a window and reports any engine errors.